GÉNÉRICITÉ EN C#

INF1035, Automne 2023

DMI-UQTR

INTRODUCTION

- Les génériques ont été ajoutés à la version 2.0 du langage C# et du Common Language Runtime (CLR).
- Les génériques introduisent le concept des paramètres de type dans .NET Framework.
- Ils permettent de concevoir des classes et des méthodes qui retardent la spécification d'un ou de plusieurs types jusqu'à ce que la classe ou la méthode soit déclarée et instanciée par le code client.
- Par exemple, à l'aide d'un paramètre de type générique T, vous pouvez écrire une seule classe qui peut être utilisée par un autre code client sans impliquer le coût ou le risque des casts ou des opérations de boxing à l'exécution, comme illustré ici :

EXEMPLE INTRODUCTIF

```
/ Declare the generic class.
public class GenericList<T> {
        void Add(T input) { }
}
class TestGenericList {
        private class ExampleClass {
        }
        static void Main() {
         // Declare a list of type int.
         GenericList<int> list1 = new GenericList<int>();
         //Declare a list of type string.
         GenericList<string> list2 = new GenericList<string>();
         // Declare a list of type ExampleClass.
         GenericList<ExampleClass> list3 = new
                              GenericList<ExampleClass>();
        }
}
```

QUE FAIT-ON AVEC LES GÉNÉRIQUE

- Optimiser la réutilisation de code, la sécurité des types et les performances.
- L'utilisation la plus courante des génériques est faite les classes de type collection.
- .NET Framework contient plusieurs nouvelles classes de collection génériques dans l'espace de noms System.Collections.Generic.
- Elles doivent être utilisées autant que possible, à la place de classes comme ArrayList de l'espace de noms System.Collections.
- Créer vos propres interfaces, classes, méthodes, événements et délégués génériques.
- Contraindre les paramètres pour y permettre l'accès à des méthodes.
- Obtenir des informations sur les de données génériques à l'exécution à l'aide de la réflexion.

CONVENTION DE NOMMAGE

- Contrairement à JAVA, C# conseille d'utiliser des noms de paramètres évocateurs et préfixés par la lettre T sauf si une seule lettre est assez évocatrice.

```
public interface ISessionChannel<TSession> { /*...*/ }
public delegate TOutput Converter<TInput, TOutput>(Tinpu from);
public class List<T> { /*...*/ }
```

PARAMETRES & CONTRAINTES

- Il est possible de contraindre les paramètres en C# avec le mot-clé contextuel where.
- Liste des six types de contraintes :

Contrainte	Description
where T : struct	L'argument de type doit être un type valeur. Tout type valeur, excepté Nullable, peut être spécifié. Consultez Utilisation de types Nullable pour plus d'informations.
where T : class	L'argument de type doit être un type référence. Cela s'applique également à tout type de classe, d'interface, de délégué ou de tableau.
where T : new()	L'argument de type doit avoir un constructeur sans paramètre public. Quand vous utilisez la contrainte new() avec d'autres contraintes, elle doit être spécifiée en dernier.
where T : <nom de classe de base>	L'argument de type doit être la classe de base spécifiée ou en dériver.
where T : <nom d'interface>	L'argument de type doit être ou implémenter l'interface spécifiée. Plusieurs contraintes d'interface peuvent être spécifiées. L'interface qui impose les contraintes peut également être générique.
where T : U	L'argument de type fourni pour T doit être l'argument fourni pour U ou en dériver.

PARAMETRES & CONTRAINTES

- Les contraintes s'utilisent comme suite sur les types génériques

```
class EmployeeList<T> where T : Employee, IEmployee,
System.IComparable<T>, new() { // ... }

class Base { }
class Test<T, U> where U : struct
                 where T : Base, new() { }
```

- Et comme suite sur les méthodes génériques :

```
public static void OpTest<T>(T s, T t) where T : class {
        System.Console.WriteLine(s == t);
}
```

Paramètres de type non bornés

- Les paramètres de type qui n'ont aucune contrainte, tels que T dans la classe SampleClass<T>{}, sont appelés paramètres de type non bornés (unbounded).
- Ils obéissent aux règles suivantes :
  - Les opérateurs **!=** et **==** ne peuvent pas être utilisés.
  - Ils peuvent être convertis vers et depuis System.Object ou être explicitement convertis vers tout type d'interface.
  - Ils peuvent être comparés à null.

TYPES GENERIQUES

- Les types génériques de C# fonctionnent de la même manière que les types génériques de JAVA.
  - Les classes peuvent être génériques
  - Les interfaces peuvent être génériques
  - Les struct peuvent être génériques.
- Les déclarations de généricité se font de la même manière.

MÉTHODES & DELEGUÉS GÉNÉRIQUES

- Les méthodes (et délégués) génériques fonctionnent de manière semblable aux méthodes génériques de Java à 2 différences près.
  - La déclaration du ou des paramètres génériques se fait après le nom la méthode entre < et > et les contraintes, après la parenthèse des arguments.
  - Au moment de l'appel de la méthode, on peut fournir le vrai type du paramètres s'il y'a des confusions.
- Comme en Java, l'inférence de type est possible dans le cas des méthodes génériques

MÉTHODES GÉNÉRIQUES

```
static void Swap<T>(ref T lhs, ref T rhs) {
        T temp;
        temp = lhs;
        lhs = rhs;
        rhs = temp;
}
public static void TestSwap() {
        int a = 1;
        int b = 2;
        Swap<int>(ref a, ref b);
        System.Console.WriteLine(a + " " + b);
}
```

- Le compilateur ne peut pas déduire les paramètres de type uniquement à partir d'une valeur de contrainte ou de retour.
- Par conséquent, impossible d'inférer des types avec les méthodes à 0 paramètre.

TYPE GÉNÉRIQUE VALEUR/RÉFÈRENCE

- Quand le paramètre d'un type générique est de type valeur, une nouvelle instance du générique est créée (un nouveau type)
- Quand il s'agit d'un paramètre de type référence, aucun nouveau type n'est créé.
  - Il s'agit du même comportement dans ce cas que le langage Java.
- Rappelons que Java ne supporte par les types valeurs dans les paramètres génériques
  - Java utilise l'autobxing et l'auto-unboxing.